

UD7. PROGRAMACIÓN MODULAR

1. Descomposición modular
2. Clasificación de los módulos
3. Ámbito de variables
4. Parámetros
5. Paso de parámetros
6. Estructura modular
7. Implementación de módulos en Java (métodos)
8. Notación de programa con módulos en Java
9. Ejemplo
10. Recursividad
11. Ejercicios

1. DESCOMPOSICIÓN MODULAR.

Consiste en dividir el programa en varias partes o componentes denominados **módulos**, cada uno de los cuales tiene una misión que deberá ser la más específica posible.

Cada módulo puede ser llamado por uno o varios módulos, quedando suspendida la ejecución del módulo que llama y entrando en ejecución el módulo llamado hasta que éste termina, devolviendo a continuación el control del programa al módulo que lo llamó en el mismo punto donde se efectuó la llamada.

1. DESCOMPOSICIÓN MODULAR.

Las principales ventajas de la programación modular son:

- ✓ Facilita la comprensión del problema y su resolución escalonada.
- ✓ Aumenta la claridad y la legibilidad de los programas.
- ✓ Facilita un mejor y más rápido mantenimiento.
- ✓ Permite que las diferentes partes pueden ser programadas de manera independiente e incluso por diferentes personas.
- ✓ Reduce el tiempo de desarrollo aprovechando módulos ya desarrollados previamente.
- ✓ Mejora la depuración de los programas, pues se pueden depurar los módulos aisladamente de forma más sencilla e independiente.

2. CLASIFICACIÓN DE LOS MÓDULOS.

Los módulos se pueden clasificar atendiendo a los siguiente criterios:

- En función de su situación con respecto al módulo que lo invoca.
 - **Internos:** cuando están en el mismo fichero (fuente).
 - **Externos:** cuando están en distinto fichero (fuente)
- Atendiendo al posible valor de retorno.
 - **Función:** aquel módulo cuya ejecución implica necesariamente la devolución de un resultado. En este caso, el módulo que llama a una función, debe recoger el valor que ésta devuelve en una expresión. Ej. `pow(3,2)` `Leer.dato()`
 - **Procedimiento:** aquel módulo cuya ejecución no implica, necesariamente, la obtención de un valor resultante.

`System.out.println("Hola");`

2. CLASIFICACIÓN DE LOS MÓDULOS.

- En función de cuándo ha sido desarrollado.
- **De programa:** su desarrollo se ha realizado en el programa actual.
 - **De librería:** ha sido desarrollado previamente y está contenido en ficheros de librería.

3. ÁMBITO DE LAS VARIABLES.

Es la parte del programa donde es conocida dicha variable y por tanto, donde puede ser utilizada.

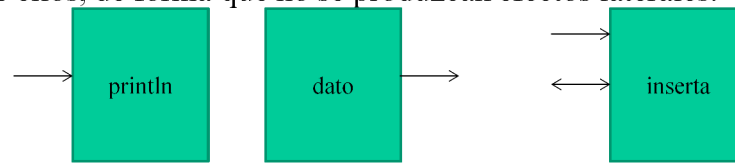
Las variables en función de su ámbito, se pueden clasificar en globales y locales.

Variables globales: deben estar definidas en un lugar específico del programa que no pertenezca a ninguno de sus módulos, o bien deben tener un identificador especial que permita diferenciarlas. Si se definen en el programa principal, su ámbito de validez es todo el programa (con todos sus módulos). Si se definen en un módulo intermedio, su ámbito de validez es ese módulo y los módulos definidos dentro de éste.

Variables locales: deben estar definidas en un módulo del programa y sólo serán visibles en ese módulo del programa.

4. PARÁMETROS O ARGUMENTOS.

Son las variables de enlace entre módulos para establecer el acoplamiento entre ellos, de forma que no se produzcan efectos laterales.



`System.out.println("Hola");` `x=Leer.dato();` `inserta(3,lista);`

Los parámetros se pueden clasificar en:

Parámetros actuales: son variables locales pertenecientes al módulo que realiza la llamada y cuyo valor (contenido) o dirección de memoria es enviado al módulo invocado.

Parámetros formales: son variables locales, pertenecientes al módulo invocado, que reciben el valor o la dirección de memoria de los parámetros actuales del módulo que lo invoca, en el momento de ser ejecutada la llamada.

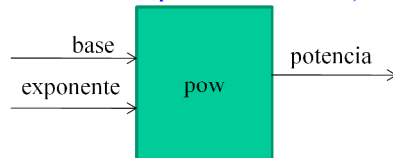
4. PARÁMETROS O ARGUMENTOS.

Los parámetros normalmente se expresarán dentro de los paréntesis que siguen al identificador o nombre del módulo.

Tiene que existir una correspondencia entre los parámetros actuales y los parámetros formales en *número, orden y tipo de datos*.

Se puede considerar como parámetro o variable de enlace, el valor retornado por una función y que será recogido por el módulo que hizo la llamada a dicha función.

`pow(parámetroActual1, parámetroActual2) → parámetroActual3`



Los parámetros podrán ser de **Entrada**, **Salida** o **Entrada/Salida**

5. PASO DE PARÁMETROS.

El paso de parámetros entre módulos se puede realizar de dos formas:

Por valor: se envía una copia del valor que tiene el parámetro actual en el momento de realizar la llamada. Este valor es recogido por el parámetro formal correspondiente del módulo invocado. Ambos parámetros deben estar definidos con el mismo tipo de datos. Por tanto, se puede operar con los valores enviados al módulo llamado, manteniendo en el módulo que realiza la llamada los valores originales en la variable que se usa como parámetro actual.

Por referencia o dirección: se envía la dirección de memoria que tiene asignado el parámetro actual. Esta dirección es recogida por el parámetro formal correspondiente del módulo invocado. Por tanto, en el módulo llamado se puede modificar el valor de la variable que se usa como parámetro actual.

5. PASO DE PARÁMETROS

Módulo pintaSuma(val x:numérico real, val y:numérico real)

Entorno:

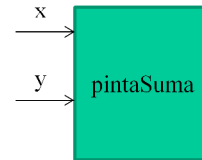
suma es numérico real

Algoritmo:

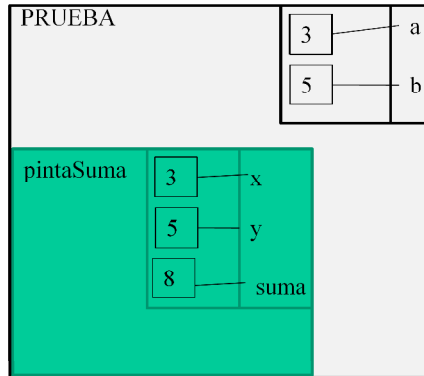
suma $\leftarrow$ x + y

Escribir "La suma es: ", suma

Fin Módulo



Número 1º: 3
Número 2º: 5
La suma es: 8



Programa: PRUEBA

Entorno:

a,b es numérico real

Algoritmo:

Escribir "Número 1º:"

Leer a

Escribir "Número 2º:"

Leer b

pintaSuma(a,b)

Fin Programa

5. PASO DE PARÁMETROS

Módulo suma(val x:numérico real,
val y:numérico real) devuelve numérico real

Entorno:

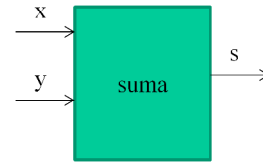
sum es numérico real

Algoritmo:

$sum \leftarrow x + y$

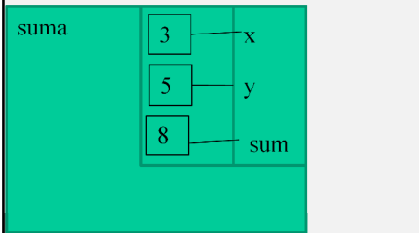
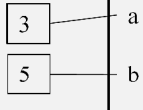
Devuelve sum

Fin Módulo



Número 1º: 3
Número 2º: 5
La suma es: 8

PRUEBA



Programa: PRUEBA

Entorno:

a,b es numérico real

Algoritmo:

Escribir "Número 1º:"

Leer a

Escribir "Número 2º:"

Leer b

Escribir "La suma es: ", suma(a,b)

Fin Programa

5. PASO DE PARÁMETROS

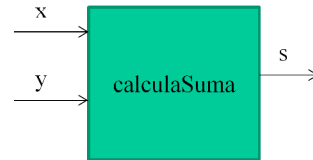
Módulo calculaSuma(val x:numérico real,
val y:numérico real,
ref. s:numérico real)

Entorno:

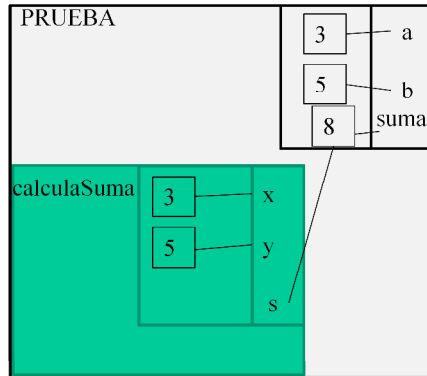
Algoritmo:

$s \leftarrow x + y$

Fin Módulo



Número 1º: 3
Número 2º: 5
La suma es: 8



Programa: PRUEBA

Entorno:

a,b,suma es numérico real

Algoritmo:

Escribir "Número 1º:"

Leer a

Escribir "Número 2º:"

Leer b

calculaSuma(a,b,suma)

Escribir "La suma es: ", suma

Fin Programa

6. ESTRUCTURA MODULAR

Módulo pintaSuma(val x:numérico real, val y:numérico real)

Entorno:

suma es numérico real

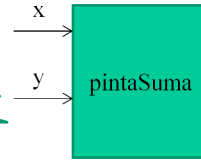
Algoritmo:

$\text{suma} \leftarrow x + y$

Escribir "La suma es: ", suma

Fin Módulo

Procedimiento



Módulo suma(val x:numérico real,
val y:numérico real) devuelve numérico real

Entorno:

suma es numérico real

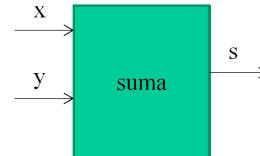
Algoritmo:

$\text{suma} \leftarrow x + y$

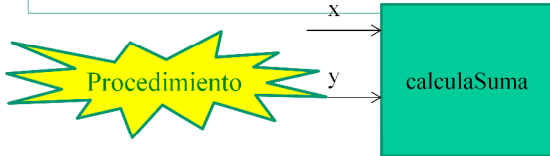
Devuelve suma

Fin Módulo

Función



Procedimiento



Módulo calculaSuma(val x:numérico real,
val y:numérico real,
ref. s:numérico real)

Entorno:

Algoritmo:

$s \leftarrow x + y$

Fin Módulo

7. IMPLEMENTACIÓN DE MÓDULOS EN JAVA (métodos POO)

```
public static tipoValorRetorno nFunción(parámetros formales){  
  //Entorno:  
    // Declaración locales  
  //Algoritmo:  
    //cuerpo del módulo  
    return valorDeRetorno;  
} //Fin Función
```

PARÁMETROS FORMALES: tipo nParam1[, tipo nParam2]

```
public static void nProcedimiento(parámetros formales){  
  //Entorno:  
    // Declaración locales  
  //Algoritmo:  
    //cuerpo del módulo  
} //Fin Procedimiento
```

Apis

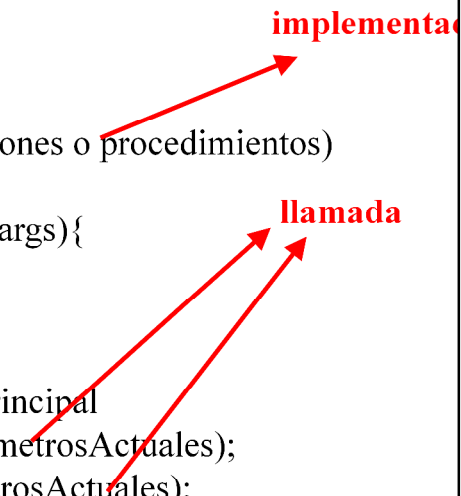
8. NOTACIÓN DE PROGRAMA CON MÓDULOS EN JAVA.

```
//Programa: nombrePrograma
//Autor: Nuria Fuentes
public class nClase{
    //Declaración de métodos (funciones o procedimientos)

    public static void main(String[] args){
        //Entorno:
        // Declaración datos
        //Algoritmo:
        //cuerpo del programa principal
        variable=nFuncion(parametrosActuales);
        nProcedimiento(parametrosActuales);
    }//Fin Programa
}
```

implementación

llamada



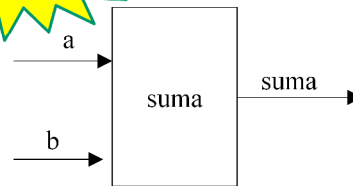
9. EJEMPLO.

```
public static void pintaSuma(float a,float b){  
    //Entorno:  
        float suma;  
    // Algoritmo:  
        suma=a+b;  
        System.out.println("La suma de "+a+" + "+b+" es: "+suma);  
    }//Fin Procedimiento
```

Procedimiento

```
public static float suma(float a,float b){  
    //Entorno:  
        float resultado;  
    //Algoritmo:  
        resultado=a+b;  
        return resultado;  
    }//Fin Función
```

Función



9. EJEMPLO.

```
//Programa: LaSuma1
//Autor: Nuria Fuentes
public class LaSuma1 {
    public static void pintaSuma(float a,float b){
        //Entorno:
        float suma;
        // Algoritmo:
        suma=a+b;
        System.out.println("La suma de "+a+" + "+b+" es: "+suma);
    }//Fin Procedimiento
    public static void main(String[] args){
        //Entorno:
        float n1,n2;
        //Algoritmo:
        System.out.print("Introduzca un n°: ");
        n1=Leer.datoFloat();
        System.out.print("Introduzca un n°: ");
        n2=Leer.datoFloat();
        pintaSuma(n1,n2);
    }//Fin Programa
}
```

9. EJEMPLO.

```
//Programa: LaSuma2
//Autor: Nuria Fuentes
public class LaSuma2{
    public static float suma(float a,float b){
        //Entorno:
        float resultado;
        //Algoritmo:
        resultado=a+b;
        return resultado;
    }//Fin Función
    public static void main(String[] args){
        //Entorno:
        float n1,n2;
        //Algoritmo:
        System.out.print("Introduzca un n°: ");
        n1=Leer.datoFloat();
        System.out.print("Introduzca un n°: ");
        n2=Leer.datoFloat();
        System.out.println("La suma es: "+suma(n1,n2));
    }//Fin Programa
}
```

9. EJEMPLO.

Ejercicio.

Hacer un programa que muestre el cubo y la raíz cuadrada de un número introducido por teclado usando **programación modular**. NO se puede usar pow.